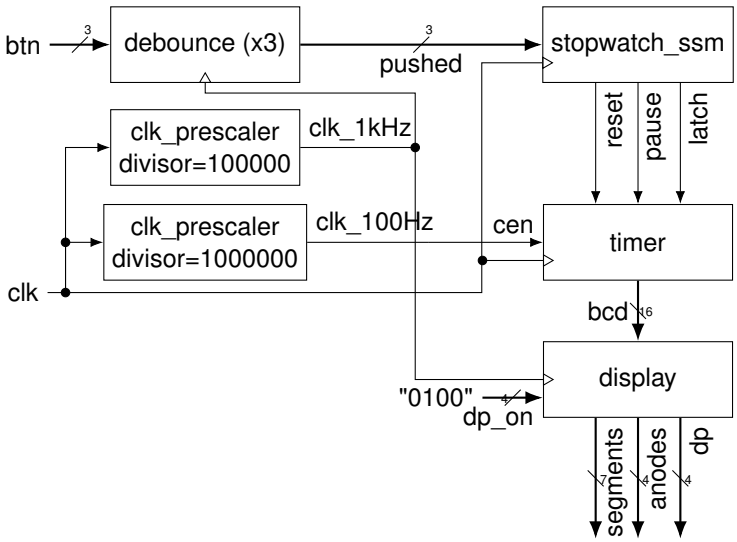
Digital Design using VHDL

Dr John A.R. Williams <J.A.R.Williams@jarw.org.uk>[1]

v2026.1.0-13-ge9cb31c



Copyright 2026 Dr. John A.R. Williams <J.A.R.Williams@jarw.org.uk>
This work is licensed under a *Creative Commons Attribution-ShareAlike 4.0 International License* (https://creativecommons.org/licenses/by-sa/4.0/).

[1]Ardenoak Consulting Ltd.

Contents

1 Overview **5**

1.1 What you will learn . . . 5
1.2 Prior knowledge assumed . . . 5
1.3 What you should be able to do by the end . . . 5
1.4 How to use this workbook . . . 5
1.5 Getting started . . . 6
1.6 Use of GitHub CoPilot AI . . . 7
1.7 Feedback based on common mistakes . . . 7
1.8 Using the Claude VHDL Tutor . . . 7
1.8.1 Setting up the tutor . . . 8
1.8.2 Working with the tutor . . . 8
1.8.3 Permissions, notes and privacy . . . 9
1.9 Formatting used in this document . . . 9

2 Introduction To VHDL Simulation and Synthesis **10**

2.1 The VHDL Digital Design Flow . . . 10
2.1.1 Creating a full adder entity . . . 10
2.1.2 Writing a test-bench . . . 12
2.1.3 Implementation on an FPGA . . . 14
2.2 VHDL entities and components . . . 16
2.2.1 The Clock Prescaler - our first synchronous circuit . . . 16
2.2.2 Debouncing and Decoding push buttons . . . 18
2.2.3 The Counter . . . 19
2.3 Putting it all together - Hierarchical design and abstraction . . . 22

3 Case Study: A Stopwatch **24**

3.1 A Stopwatch . . . 24
3.2 Seven-segment display . . . 25
3.3 The Timer and the Stopwatch Control Logic . . . 28
3.3.1 Stopwatch control state machine . . . 28
3.3.2 Timer logic . . . 29
3.4 Putting it all together - the top level design and synthesis . . . 29

A Some VHDL Style Guidelines **31**

Chapter 1

Overview

This workbook introduces the design of synchronous digital circuits using VHDL and the FPGA design flow. It is intended for undergraduate students and for independent learners who already know some basic digital electronics but are new to hardware description languages. You will work through a series of practical exercises that begin with simple combinational logic and build towards a complete stopwatch design. It is intended that you will carry out the activities in this book in parallel with watching the supporting introductory VHDL videos.

1.1 What you will learn

By the end of this workbook you should be able to:

- read and write simple synthesizable VHDL;
- design and test combinational and sequential circuits;
- use a state-machine approach to describe synchronous behaviour;
- write testbenches and use simulation to check a design;
- synthesize and implement a design for an FPGA.

1.2 Prior knowledge assumed

The workbook assumes that you already know:

- basic digital logic concepts such as AND, OR, NOT, XOR, truth tables and Boolean algebra;
- simple sequential concepts such as clocks, flip-flops and registers;
- how to use a text editor and a terminal.

You do not need prior experience of VHDL.

1.3 What you should be able to do by the end

By the end of the course you should be able to design, simulate and synthesize a synchronous digital circuit using a state-machine approach, and to understand how that design can be implemented on an FPGA.

1.4 How to use this workbook

This workbook is designed to be used in order. The main learning path is:

Essential the introduction, the guided exercises, the simulation and synthesis sections, and the stopwatch case study.

Extension material advanced exercises, deeper discussion of design trade-offs, and the appendices.

If you are working independently, begin with the core exercises and only use the extension material once you are comfortable with the basic ideas. If you get stuck, use the hints and warnings in the text, revisit the earlier examples, and consult the appendix and the provided VHDL templates.

You will be implementing and testing separate components as you go along and then combining them at the end to produce a finished complete working system. As you go through the individual exercises the amount of additional hints and support provided here decreases, and you will be expected to be able to find things out for yourself.

As part of this process your will gain familiarity with:

1. using a text editor to edit and save your source files;
2. the VHDL tool chain;
3. writing test-benches and using simulations to test your digital designs;
4. using synthesis tools to generate bit files to program FPGAs.

The exercises in this workbook require that you have installed the AMD Vivado Design Suite[1] and Microsoft Visual Studio Code[2] and that you are using the Basys3 Artix-7 FPGA Board[3]. Other boards using AMD FPGAs and with equialent features can be used however you will need to modify the constraints (xdc) files to map the device pins to the VHDL signals, and change the partnumber specified in the file "Taskfile.yml".

You may use a recent Windows or Linux system supported by the Vivado design suite. The advantages of using the Linux command line interface compared to using an integrated development environment (IDE) which you may previously have used for software development are:

1. the command line tools are ubiquitous and widely used;
2. the tools are generally more mature - tried,tested and proven;
3. it is faster and more robust;
4. you can use many of the same tools across different types of projects - you don't have to learn a different set of editor commands for every programming environment;
5. it is easier to automate many tasks;
6. it is therefore more efficient for larger complex systems.

Many of these skills are transferable to writing software code and so this practical should be considered as general training to developing software as well as hardware.

Both a Taskfile and a more traditional Makefile (Linux only) are provided to automate the analyzing, testing and synthesis of your designs and to hide many of the complex details of the Vivado tool-chain can be hidden allowing a focus on VHDL design in this course. The more modern Taskfile provides better integration with VS Code via the Task extension which provides a list of the key tasks in the Explorer window, and supports both Linux and Windows consistently and is now preferred, however makefiles are potentially more powerful and flexible.

While the Vivado GUI IDE supports more advanced features such as live debugging, and optimizing and visualizing timing and power handling these don't apply in this introductory course. The proprietary Vivado gui is not well-designed and is very complex to learn to use. .

1.5 Getting started

The exercise templates, testbenches and build files for this workbook are in the LearnVHDL repository on gitHub[4]. Clone this repository to get your own working copy, for example using the option "Clone Git Repository" in Visual Studio Code. I recommend using git (and gitHub) to manage version control of your work. I also recommend using Visual Studio Code to edit and manage your work as this provides good support for VHDL as well as other languages and integrates easily with git and gitHub. These are all modern industry standard tools for software development.

- Watch the video on how to edit files, build,program and debug using the Basys 3 FPGA board.

[1] https://www.amd.com/en/products/software/adaptive-socs-and-fpgas/vivado.html
[2] https://code.visualstudio.com/download
[3] https://www.digikey.co.uk/en/product-highlight/d/digilent/basys3-artix-7-fpga-board
[4] https://github.com/willijar/LearnVHDL

1.6 Use of GitHub CoPilot AI

I have provided a CoPilot Code Reviewer workflow in your GitHub repository which can check your code for good practice and for similarity to code found on the internet. To enable this open the VS Code command palette (Ctrl+Shift+P), select “Chat: New Mode File” and choose the file “.gitHub/chatmodes/Code reviewer.chatmode.md”. I would recommend that you enable this workflow in your repository and use it to check each file after you havecompleted and tested your solution to get feedback on style and possible improvements. If the workflow finds any issues with your code, you should consider and if need be address them.

The free version of CoPilot is limited in the amount of code it can generate and may not be able to generate complete solutions for you. Additionally, you should not rely solely on CoPilot to generate your code as it may not always produce correct or optimal solutions. The amount of usage of the free version is limited. You may need to purchase a subscription if you use it extensively.

I strongly recommend your turn *off* CoPilot completions as these are not always right and will distract and hinder your learning. Once you are more experienced and can easily and quicly judge the AI suggestions you may find it useful to turn these on again.

1.7 Feedback based on common mistakes

- You must configure the constraints file to map the top level ports to the hardware pins. Otherwise, the circuits will not synthesize.
- You must implement entities as specified - using exactly the entity names and port and generic names and types as specified.
- The use of two **for** ... **generate** statements in your fourcounter implementation would be more idiomatic and would remove duplication. While the reduction here is not substantial here it would be more important in larger circuits.
- Not providing correct down_event **signal** in debounce entity . This needs to be 1 for 1 clock pulse only when debounce goes from high to low. To determine this you will need to use an internal signal (say debounce_i) to latch the debounce output so that you can test the previous value in the clocked process.
- No need to use and gate entity in fourcounter implementation as AND is an inbuilt function in VHDL. The initial and gate entity is simply an example to get students started using Vivado.

1.8 Using the Claude VHDL Tutor

Your copy of the exercise repository includes a VHDL tutor which runs in a Claude Code[5] session, in a panel inside Visual Studio Code. I have configured it to help you learn rather than to do the work for you. It asks you questions and gives hints in stages, but it will not write your solution.

The tutor knows the exercises in this workbook, the coding rules your work should follow, and the commands for simulating and synthesizing your designs. It can also point you to the part of a supporting video which explains an idea you are stuck on. While you work it can:

- look at your files and run the analyzer or simulator to see the same messages that you see;
- ask questions which lead you towards the cause of a problem;
- check that you understand a fix by asking you to explain it or predict what would change;
- review your code when a testbench passes, and raise any issues one question at a time;
- keep notes on your progress so that it can pick up where you left off next time.



The tutor can be wrong. As with any generative AI, check what it tells you against this workbook, the simulator and your own reasoning. You remain responsible for your design.

[5] https://code.claude.com/docs/en/vs-code

1.8.1 Setting up the tutor

You will need a paid Claude plan (such as Claude Pro[6]) or a Claude Console account with API credit, because the free Claude plan does not include Claude Code. The tutor is optional and none of the exercises depend on it. Set up the rest of your tools first, as described in section 1.5. Then:

1. Check that the folder `.claude/agents` in your repository contains the file `tutor-agent.md`. If it does not, you have a version of the exercise repository which does not include the tutor, so you will need to get a newer version (see section 1.5).
2. In Visual Studio Code (version 1.94 or later) open the Extensions view (Ctrl+Shift+X), search for "Claude Code" and install the extension published by Anthropic. You do not need to install anything else to run Claude Code. On Windows you should also install Git for Windows[7] if you have not already done so.
3. The tutor uses `pdftotext` to look up the exact wording of an exercise. On Debian or Ubuntu Linux you can install it with

   ```
   sudo apt install poppler-utils
   ```

   This step is optional, as the tutor has its own summary of each exercise. If it needs the exact wording and cannot read it, tell it which exercise you are on and what the exercise asks for.
4. Open the top folder of your repository in Visual Studio Code ("File" then "Open Folder"). If you are asked whether you trust the folder, say yes, as the extension does not work in Restricted Mode.
5. Open the Claude Code panel by clicking on **Claude Code** in the status bar at the bottom right of the window, or on the Spark icon in the toolbar at the top right of an open file.
6. The first time you do this click on **Sign in** and complete the authorization in your web browser.
7. Type "Hi" or something similar in the panel and press Enter. The tutor does not start until you send a first message, and you will need to do this each time you start a new conversation.

The tutor will then look at your most recently changed VHDL file, greet you with a comment on your work and ask you a question.



Always open the top folder of your repository, not a parent folder or a single file. Otherwise you will get the general purpose assistant, without the tutor's knowledge or its rules. If you do not get the greeting described above, check which folder you have open.

1.8.2 Working with the tutor

Keep the panel open while you work on your design. Type your questions in plain English. The tutor can see which file you have open and any text you have selected, so you can select a few lines and ask about them directly. You will get the most from it if you say what you have tried and what you expected to happen, for example:

- "I'm starting exercise 4. I don't see how to work out how many bits the counter needs."
- "My counter testbench fails at 40 ns. I expected the count to be 3 but it is 2."
- "What does this analyzer error on line 23 mean?"
- "Why should I use a clock enable rather than a slower clock?"

Expect to be asked a question in return, and try to answer it even if you are unsure, because your answer shows the tutor what to explain next. If you are still stuck, say so and it will give you a more specific hint, point you to part of the workbook, or suggest a video.

The tutor will not write your architecture, process, testbench or constraints for you, even if you ask, although it will remind you of a line of syntax such as rising_edge(clk). Occasionally you may see it say too much and then correct itself with a follow-up question. This is an automatic check on its replies.

I recommend the following workflow for each component:

1. Read the exercise and make your own attempt without using AI.
2. Analyze your code and read the messages yourself before you ask for help.

[6] https://claude.com/pricing
[7] https://git-scm.com/downloads/win

3. Use the tutor when you are stuck, and make the changes to your code yourself.
4. Run the testbench using `task` in a terminal, for example `task counter_testbench`. If a tutor session is open and your testbench passes, it will be told automatically and will start a code review.
5. Work through the tutor's questions and decide for yourself which changes to make. A passing testbench does not prove that your design is correct, so think about which cases it does not check.
6. Commit your work with git.

1.8.3 Permissions, notes and privacy

The tutor is not able to edit your VHDL, constraints or build files. It may ask your permission to run a command, such as an analysis or simulation using `task`. Read each command before you approve it, and decline anything that you do not recognize or that would change your files. Leave the panel in its Manual permission mode so that you are asked. Programming the board is left to you.

The tutor keeps its notes on your progress in the folder `.claude/agent-memory-local`. These are not committed to git, so they stay on that computer and a tutor on another machine will start afresh.

A few controls in the panel are useful:

New conversation Choose "Claude Code: Open in New Tab" from the command palette (Ctrl +Shift +P) when you move on to a new exercise.
Session history This button at the top of the panel returns you to an earlier conversation.
Stop or Esc This interrupts the tutor if it is going in the wrong direction.

Each use of the tutor counts against the usage limit of your plan (or your Console credit), and long conversations use more. Starting a new conversation for each exercise helps with this.



Your files and messages are sent to Anthropic for processing when you use the tutor. Do not put passwords or personal information in your conversations or repository, and review the privacy settings of your Claude account.

Optional Exercise. Meet your tutor

Once you have set up the tutor, open the panel in your repository and tell it which exercise you are on. Ask it a question about something in that exercise which you are not sure of, and then try asking it to just write the solution for you to see how it responds.

1.9 Formatting used in this document



Warning will be formatted like this. These blocks provide important information which you must follow otherwise your design may fail.



Block formatted like this provides useful explanatory information. While this information is not absolutely necessary to know to complete the immediate task it will help you understand why things are done a particular way. This will help you in your future work.

Exercise 1. An Exercise

The instructions for the exercises you have to complete are formatted like this. Make sure you follow all steps.

```
# commands which you must be type in a command shell is formatted like this
```

```
-- vhdl fragments will use colour syntax highlighting in the text with
-- red for comments, and blue for keywords e.g. as here
-- the library lines below will often be left out in the example fragments
-- in this document as they
library ieee ;
use ieee.std_logic_1164.all;
use IEEE.numeric_std.all;

-- vhdl entity declaration goes here
-- followed by one or more corresponding architecture definitions.
```

Chapter 2

Introduction To VHDL Simulation and Synthesis

2.1 The VHDL Digital Design Flow

At the end of this section you will learn:

1. the VHDL design flow;
2. basic shell commands you will need to use on the Linux command line;
3. how to copy, rename and edit files in Linux;
4. how to use a provided `Makefile` and the `make` utility to control the execution of a complex sequence of commands on demand.

The basic design flow in VHDL is as follows:

1. think about a design you want to implement, e.g. an adder,
2. implement the design in VHDL,
3. implement a test-bench in VHDL,
4. use the test-bench to simulate and test the design (from 2),
5. if this works and the simulation is successful, try to synthesize the design,
6. map, place and route the design for a particular FPGA,
7. build a .bit file,
8. program your FPGA (using USB, JTAG or similar to upload bit file),
9. finally test you design in hardware - possibly using logic analyser or similar.

2.1.1 Creating a full adder entity

In the first practical you will implement a simple combinational circuit, a full adder, and test it going through steps 4–8 using the provided tools. You will test the adder circuit in simulation and may also synthesize and implement it on the Basys-3 FPGA board.

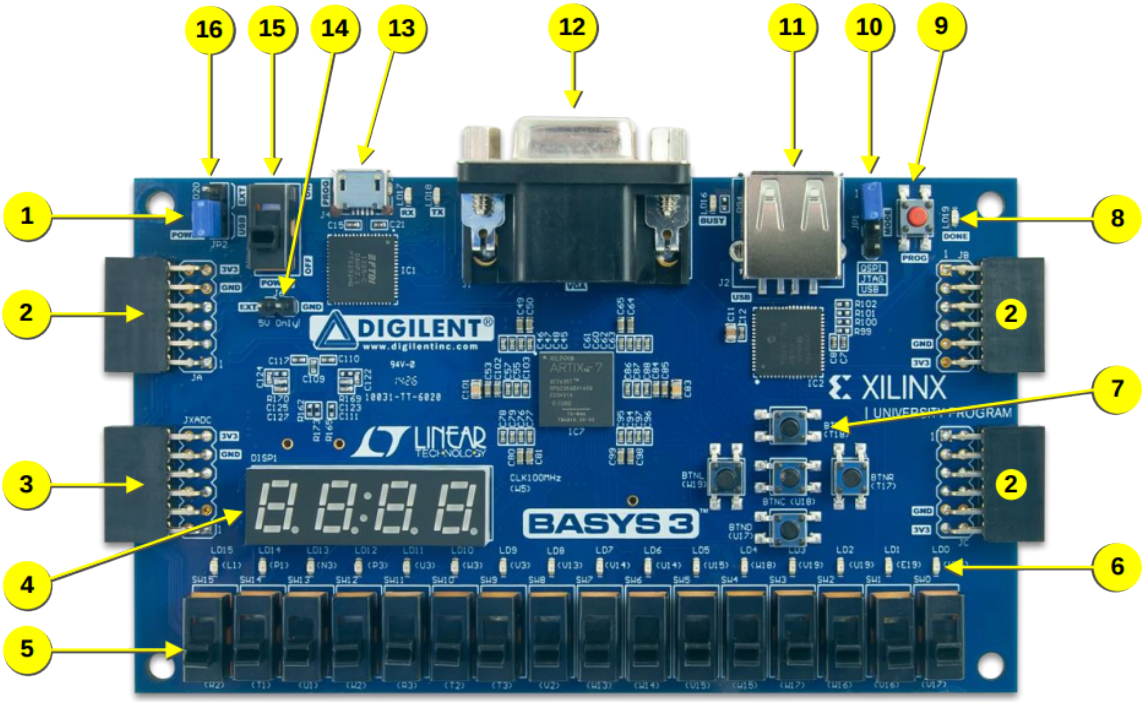
The binary full adder which has three inputs, Carry in **cin** and two input bits **x** and **y** and two outputs sum **s** and carry out **cout** (see figure 2.2).

Exercise 1.

1. Once you have cloned your repository from gitHub (see section 1.5) open it you should see a directory list in Visual Studio Code which includes the following files:

   full_adder.vhd A template for you to complete for your full adder entity and behavioural architecture.
   testbench_template.vhd The VHDL template for a simulation test-bench.
   Makefile The VHDL design flow rules and scripts for the make utility
   Basys3_Master.xdc A master constraints file for you to copy and modify for your own designs. The synthesis constraints file maps buttons and LEDs etc on the hardware to the top level entity ports.

2. View each file in turn. Identify



Callout	Component Description	Callout	Component Description
1	Power on LED	9	FPGA configuration reset button
2	Pmod port(s)	10	Programming mode jumper
3	Analogue signal Pmod port (XADC)	11	USB host connector
4	Four digit 7-segment display	12	VGA connector
5	Slide switches (16)	13	Shared UART/JTAG USB port
6	LEDs (16)	14	External power connector
7	Pushbuttons (5)	15	Power switch
8	FPGA programming done LED	16	Power select jumper

Figure 2.1: Basys 3 FPGA board

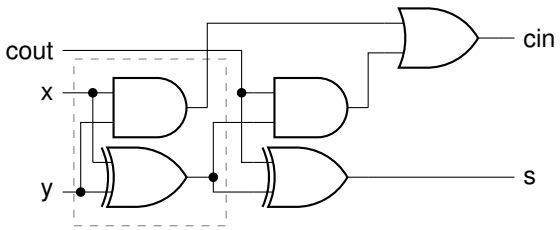


Figure 2.2: Circuit for a Binary Full Adder

(a) the full_adder **entity** declaration and behavioural architecture definition;
(b) the lines in the `Makefile` which list which source files to use when simulating or synthesizing the design. You will need to add similar lines in future for your own designs;
(c) the lines in the constraints file which map the signals to the device pins. You will need to copy and amend the master constraints file `Basys3_Master.xdc` for your own designs in future. The constraints file should have the same prefix name as your top level vhdl file i.e. if your top level vhdl file is `<name>.vhd` then you will need to create a copy of `Basys3_Master.xdc` as `<name>.xdc` to modify.

3. From the circuit 2.2 for the full adder derive the logic expressions for the two outputs as a function of the three inputs

$$\begin{aligned} \mathbf{s} &\Leftarrow \\ \mathbf{cout} &\Leftarrow \end{aligned}$$

4. In you working directory open the file called `full_adder.vhd` in an editor and in the VHDL architecture body implement the logic expressions for the outputs from the logic expressions you derived previously.



Even though VHDL is case-insensitive, filenames are case-sensitive in Linux. You must *use the exact file names given (including the correct case) and they must match the VHDL entity name in these exercises.*

5. Save your modified VHDL file `full_adder.vhd` and check that the syntax is correct by pressing Ctrl-Shift-B from within the file editor (configured in `.vscode/tasks.json` for build) or on the commandline using task

```
task VHDL:analyze:full_adder.vhd
```

or using make

```
make full_adder.check
```

This will run `xvhdl` to analyze the file and print out errors (which will also appear in the problem terminal window). If there are any errors then correct them, save your amended file and check again. It is important that you learn to understand the error messages and to use them to correct mistakes in your VHDL syntax.



VHDL has in-built Boolean operators e.g. **and**, **or**, **xor** etc and arithmetic operators +, − etc. It therefore isn't necessary to create new entities such as the one above in your future designs. Use the in-built operators instead.

2.1.2 Writing a test-bench

To check the functionality we use simulation and test-benches to check that the entity behaves as expected.

Exercise 2.

1. Complete the truth table below for the full adder.

cin	x	y	s	cout
0	0	0		
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		

You will use this truth table to create a test-bench for your full_adder VHDL entity.

2. In you working directory create a file `full_adder_testbench.vhd` and open in an editor. You may wish to copy the provided template `testbench_template.vhd` to use as a starting point.

3. Declare the full_adder_testbench entity with no inputs and outputs. This is required for a test-bench which will operate as a top level simulation entity.

   Define an architecture for the full_adder_testbench entity. Typically, this has the following form where uut refers to the name of the unit (VHDL entity) under test.

```
architecture testbench of uut_testbench is
  component uut is
    port (  -- copy ports from uut entity declaration here
      );
  end component;

-- other declarations needed for test go here e.g. signal declarations

begin
  -- entity instantiation with port maps goes here
  top: uut port map (
   -- map uut ports to local signals here
    );

  stim_proc : process
  begin
    -- test bench behaviour goes here

    -- stop process when finished test
    wait;
  end process;

end testbench;
```

4. Complete the component declaration in the architecture by using the entity name and its port names exactly as you specified them in the full adder entity declaration in `full_adder.vhd`. You can copy and paste the port declaration.

5. While we could declare separate named signals for each input and output port of the full adder it is more convenient to use vectors (arrays) of logic bits so that we can assign inputs and test outputs in one statement rather than several in our test-bench.

   For this test-bench you use two logic vectors - one for the inputs to the full adder and one for the outputs. We are then going to map these to the individual signal names on the adder (unit under test or uut).

   The syntax to declare your signals in the architecture is as follows

```
signal input  : std_logic_vector(2 downto 0);
signal output : std_logic_vector(1 downto 0);
```

   These declare signal vectors of 3 and 2 bits which can be indexed by a number.

   Bit order: When used in VHDL vectors **downto** or **to** specify the direction of the ranges. **downto** corresponds to little endian order where the least significant bit of an integer is stored in the lower position. This is the most common order used in VHDL, and it is strongly recommended that you use this order in your designs to ensure compatibility with most libraries and packages. It is important not to mix orders in your VHDL as it can lead to confusion and errors.
   Occasionally it is necessary to use a reversed order in interfacing with other entities (such as PowerPC architecture processors). In that case you will need to reverse your bits at the interface.

   And when you instantiate the full adder for testing map the signals to the inputs and outputs are declared as follows.

```
top: full_adder port map (
  x => input(0),
  y => input(1),
  cin => input(2),
  s => output(0),
  cout => output(1)
  );
```

6. You can assign values to all the inputs and test all the outputs using vectors of bits rather than setting each bit individually as was the case for and_gate_testbench.

   Add the line

   ```
   input <= "000"; wait for 10 ns; assert output = "00" report "0+0+0 failed";
   ```

   at the start of your test bench process. This assigns 000 to the input signal. Since the input bits are mapped to the ports in the full_adder we are setting all the inputs in one expression. The test bench process then has a short delay. This is to allow for the input signals to propagate through the entity under test and for the output to settle. The outputs from the test entity are mapped to the output array so the test **assert** output = "00" tests all the bits in one expression. It corresponds to testing the first line of the full adder truth table you derived previously.

7. Add similar lines to check all 8 rows of your truth table and save the `full_adder_testbench.vhd` file.

   Check the syntax by pressing Ctrl-Shift-B to analyze the file.

   If there are any errors correct them, save the amended file and check again until there are no syntax errors.

8. Run the test-bench simulation either by running full_adder_testbench or on the commandline

   ```
   task full_adder_testbench
   ```

   What output does it generate? Is your adder working correctly?

2.1.3 Implementation on an FPGA

Once we are sure out design is functioning correctly and passes the simulation test-benches we can use it to program the hardware.

In the VHDL for our full adder we gave the input and output signals the names cin, x, y, cout, s. To show this circuit working in hardware we need to map the signal names to the actual physical pins. A constraints file is used to describe this mapping. We have provided a template constraints file `Basys3_Master.xdc` which you can copy and edit to use in your own projects. This has listed and commented out all the inputs and outputs that you have available on the Basys 3 prototyping board. The build rules implemented in the `Makefile` provided require that this file must have the same name as the top level entity name (which also must match the top level vhdl file in which it is contained) that is to be synthesized.

Exercise 3.

1. Make a copy of the provided constraints file with the name `full_adder.xdc` and edit it to map the VHDL signals to the relevant pins connected to the switches and LEDs on the board.

   ```
   cp Basys3_Master.xdc full_adder.xdc
   ```

2. In these constraints file you will see lines like

   #set_property PACKAGE_PIN V17 [get_ports {sw[0]}]

   This line maps the pin V17 on the chip to VHDL signal sw[0] which is the 0th element of vector signal sw. Lines starting with a '#' are comment out in this file and will be ignored by the tools.

   Uncomment these lines for the pins which you need and change the VHDL signal names to match those in your design. The file has comments indicating what each of the sets of pins are and the constraint lines for the switches and LEDs are in order.

3. Run the synthesis tools to generate a `bit` file for the adder circuit. full_adder.bit or

   ```
   task full_adder.bit
   ```

   Note if this fails you may need to change the path to Vivado embedded in the Makefile to point to the current version installed on your system.

   View the output from the Vivado application as it runs. If there are any errors then there is something wrong with either your design or the constraints file.

   Examine all the files in the log subdirectory. Do you understand what they are telling you? Ask your lab demonstrator about them.

4. Plug your Basys 3 board into a USB port, make sure the jumper JP1 is set for USB programming and that it is switched on and either run program_full_adder or type

   ```
   task program_full_adder
   ```

   This will use the tools to upload the previously generated bit file to the board. You should see the programming light go orange and then turn off when it has finished the programming.

5. Use the switches you selected in the constraints file to change the inputs and view how the output LED changes. Is it operating as expected for a binary adder?

2.2 VHDL entities and components

At the end of this practical you will learn how to:

1. represent a clocked entity in VHDL;
2. build a clock prescaler to drive low frequency clocked circuits;
3. understand the issue of clock skew and the importance of using clock enable signals to mitigate its effect;
4. what switch bounce is and build a digital debounce circuit using VHDL;
5. build a synchronous counter and understand why it is better than a ripple counter design;
6. use a VHDL test-bench for a clocked circuit;
7. capture and display response wave-forms from the simulator;
8. validate the function of the circuit by examining the simulation results;
9. implement and test your circuit on an FPGA.

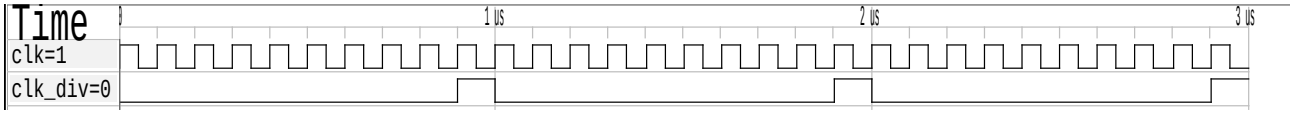
2.2.1 The Clock Prescaler - our first synchronous circuit

Prescalers are used to slow down clock signals. A clock with a frequency of "f" goes into the input, and you get a lower frequency coming from the output. In this section, we'll make a general purpose prescaler which we can then use to drive counters at a low frequency (so we can see them count), to drive output devices (such as the 7-segment display) and to control the rate at which we sample input signals (such as from buttons).



Figure 2.3: Clock Prescaler

The circuit must output a pulse of a single system clock duration every N input clock cycles as shown in the figure below for N=10.



Exercise 4.

1. Complete the definition VHDL architecture for the clock prescaler entity in the provided template file `clk_prescaler.vhd` as defined below

```
entity clk_prescaler is
  generic (
    n: integer                    --  divide clk by n to generate clk_div
  );
  port (
    clk: in std_logic;            -- input (system) clock
    clk_div: out std_logic:='0'   -- clock enable signal at clk/n frequency
    );
end clk_prescaler;
```

2. Press Ctrl-Shift-B or use the command

```
task VHDL:analyze:clk_prescaler.vhd
```

   to check your syntax as you work. This entity should output a single pulse on clk_div output every divisor clock cycles of the system clk input. The output pulse should have the width of the system clock period. Make sure to count the numbeer of clock cycles per output pulse.

3. We have provided a testbench `clk_prescaler_testbench.vhd` to simulate test the functionality of your circuit. As usual use clk_prescaler_testbench or

```
task clk_prescaler_testbench
```

   to run this test. If there are any errors reported correct your code and test again.



It is recommended that, for simulation purposes, tests are done over small samples - here we are dividing the clock by 10. With a large divider ratio (such as 100000 to reduce 1 MHz to 1 kHz) the simulations will take a very long time, and if you produce a waveform file for viewing it fill your disk space or become too big to be usable.

4. Examine the file `clk_prescaler_testbench.vhd` to see if you can determine how it works. How would you change it to divide the simulated system clock by 12?

5. In the provided template the number of bits needed for the counter is calculated depending on the divisor n. Make sure you understand this calculation - if you do not ask a lab demonstrator to explain. What libraries do we need to include to perform this calculation?

   What are the advantages be of using the IEEE unsigned type for this counter rather than the inbuilt VHDL integer type?



Clock Generation and Clock enable signals It is common in digital design that different circuits may be required to run at different clock rates. In this exercise, for simplicity, we have created a new clock by dividing down the system clock. However, in register transfer circuits all registers should be triggered from the same clock domain with the minimum of skew between stages. Routing clock signals through logic adds skew to the signal any may result in glitches which makes such signals unsuitable as clock triggers. The solution is to use a clock enable input which is driven by a clock divider to be true for a fraction of the global clock cycles. This then enables the circuit at the necessary clock rate while still triggering the circuit on the master clock edge. Synthesis tools will recognize this idiom and automatically use the built-in clock enable circuitry in the hardware.
In vhdl a clock enable signal is used in a clocked process as follows

```
SYN_PROC : process(clk)          -- clocked transition process
begin
  -- asynchronous reset may go here
  if(rising_edge(clk)) then
    -- synchronous reset may go here
    if clk_en='1' then
      -- enabled synchronous process here
    end if;
  end if;
end process;
```

where **clk_en** is the clock enable signal and clk is the system clock.
Using a clock divider to produce a new clock domain can cause timing issues or clock domain crossing problems such as clock skew, meta-stability, data loss, and data in-coherency. To avoid the clock domain crossing issue, it is better to generate a slow clock enable signal instead of creating another slower clock (using clock dividers) to drive another logic part of your design. If you really do need to run multiple clock domains then you will need to use multi-flip-flop synchronizers to synchronize the inputs from another clock domain.



The outputs of combinational logic should only be enabling signals and should never be used as clock signals. Only system clocks should be used as clock signals in your synchronous designs.

There are additional advantages of using a clock enable when using FPGAs. FPGAs have a limited number of clock sources and a limited number of global low-skew clock trees for distributing global clock signals to blocks. Therefore, the number of global clocks should generally be kept to a minimum. Synthesis tools can recognize a clock enable signal and will route it to the clock enable inputs on the registers (if the FPGA architecture has them) rather than through combinational logic.
Modern FPGAs have special clock management circuits that should be used where multiple clocks are needed however these are beyond the scope of this module, so we will be using our clock prescaler to provide low frequency clocks to interface with input and output circuitry.



Global Resets You will notice that I have not included a global asynchronous reset input in the clock prescaler entity. Whether this is needed is a matter of debate. Xilinx recommend having only local resets where necessary in your designs and not having a global reset which would use up connectivity. FPGAs are always globally reset on initialization and additionally in simulations all values will also be set at the start. It is therefore not necessary here however if implementing on a different architecture it may be necessary to include global resets.

2.2.2 Debouncing and Decoding push buttons

Now that we have our clock prescaler we are going to use it to sample input buttons at a low frequency (1 kHz) so we can both debounce them and generate synchronized signals that we can then use in subsequent circuits.

Bouncing is the tendency of any two metal contacts in an electronic device to generate multiple signals as the contacts close or open. Debouncing is used to ensure that only a single signal will be acted upon for a single opening or closing of a contact. This can be done with some analogue electronics or digitally. Digitally the principle is to make sure that the output of the switch has the same value for a particular period of time longer than the duration of any bounce in the signal. In this laboratory will be using the clock prescaler circuit to provide a 1 kHz sample clock to measure the state of a button at 1 ms intervals and check for 3 consecutive samples begin the same. This is long enough to debounce the buttons and switches on the Basys 3 boards however some other switches might bounce for longer than 1 ms and might need a longer sample period more samples.



In the implementation I am asking you to complete here you are not just debouncing the input button but also ensuring that the debounced signal is synchronized to the system clock.
The process of synchronization of asynchronous inputs (such as signals from our buttons or an independently clocked connected circuit) to a synchronous digital circuit is necessary to ensure that the signals going into a flip-flop don't change between its setup and hold times which would give rise to metastability issues. Although we are designing for an external input here synchronization techniques (which might include FIFO buffers) are also needed when connecting signals between circuits running at two different clock frequencies (between clock domains).

Exercise 5. Debouncing using VHDL

1. Complete the VHDL architecture for the debounce entity in the provided template file `debounce.vhd` as defined below.

   This entity should debounce a switch input. It should sample the input using low frequency strobe clock pulses and outputs the debounced logic value if the input is stable for longer than the bounce timescale which is three consecutive samples. It should also generate a `down\_event` pulse for one system clock pulse duration if the output changes from 0 to 1.

```
entity debounce is
  port (
    clk: in std_logic;     -- system clock
    clk_en: in std_logic; -- low frequency sampling clock enable (e.g. 1kHz)
    button: in std_logic;     -- input from button
    debounced: out std_logic:='0'; -- the debounced button level output
    down_event: out std_logic:='0' -- enable event signal when button pressed
    );
end debounce;
```

   You will need to store the two previous samples and compare them with the current sample to determine if the input is stable. You can then shift along the samples, adding in the new sample and discarding the oldest one for the next sample event.

   This is an example of using a clock enable signal to clock entities at a lower frequency while still using the master clock to control timing.

2. Use the testbench `debounce_testbench.vhd` to simulate test the functionality of your circuit.

```
task debounce_testbench
```

   to run this test. If there are any errors reported correct your code and test again.

You have created the entities necessary to provide clean input signals for controlling your circuits. The push buttons provide a 1 signal when depressed and a 0 signal otherwise. The event signal provides a single clock enable pulse when the button every time the button is pressed.

2.2.3 The Counter

Counter Types

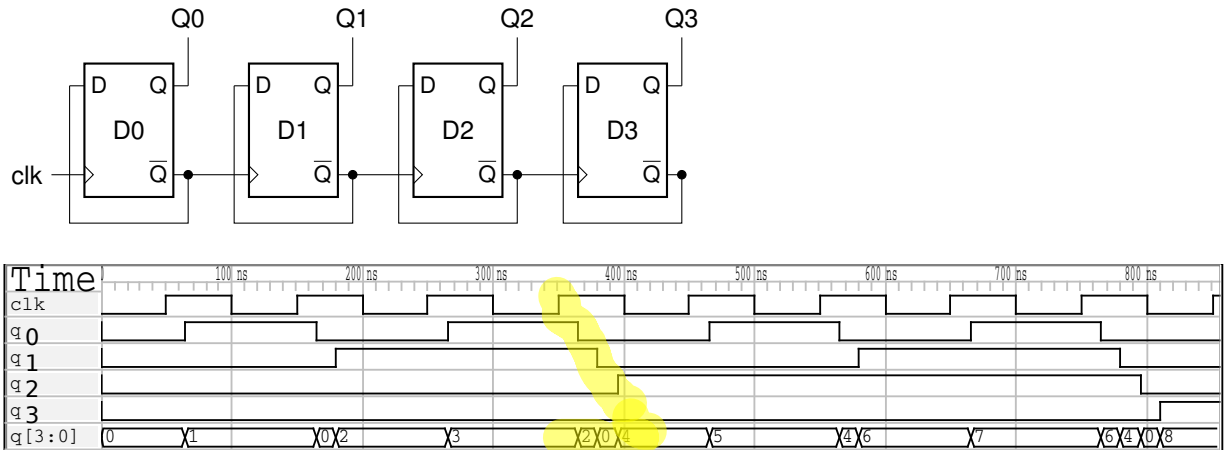


Figure 2.4: A 4-bit Ripple Binary Counter

A Ripple Binary Counter In figure 2.4 we show the schematic of a classical 4 bit binary ripple counter constructed using D-type flip-flops. Each flip-flop is in a divide by 2 arrangement (inverted output used as input for next clock cycle). The 0th digit is driven by the counter clock. Every time it changes from 1 to 0 the inverted output $\overline{Q}$ (overflow or carry bit) changes from 0 to 1 which drives the clock input of the next digit resulting in it changing state. And so on. If the delay between clock transition and change in output of each flip-flop is Δt then we can see that for the n'th bit the change will be $n\Delta t$ as it the signal propagates up the sequence of flip-flops - the output bits do not change all at once but instead the change ripples along the sequence of bits - hence the name. The counter output is not valid until all the bits have changed to their final state which depends on the number of bits - this may be seen in the highlighted region in the timing diagram. We need to make sure that the output is only read in subsequent circuits after the count has fully propagated. This can make using such a counter in subsequent circuits problematic as the timing skew must be accommodated.

A Synchronous Binary Counter An alternative approach to the ripple counter is the synchronous counter where all the flip-flops are driven from the same clock and hence will change at the same time. Such a design is much easier to use in a synchronous digital circuit - the timing is more accurately controlled and does not change depending on the size of counter.

Each stage in this counter has an output which is true when it is at its terminal count(TC) i.e. when all less significant bits are at 1 - 1 clock cycle before the counter overflows. This used as the clock enable input of the next stage so that upon the next clock cycle the next bit will change state. All the counter stages will change state at the same time and there is no accumulated delay or ripple in the change of the bits. We can cascade these counters without worrying about the accumulation of delay or skew in the clock.

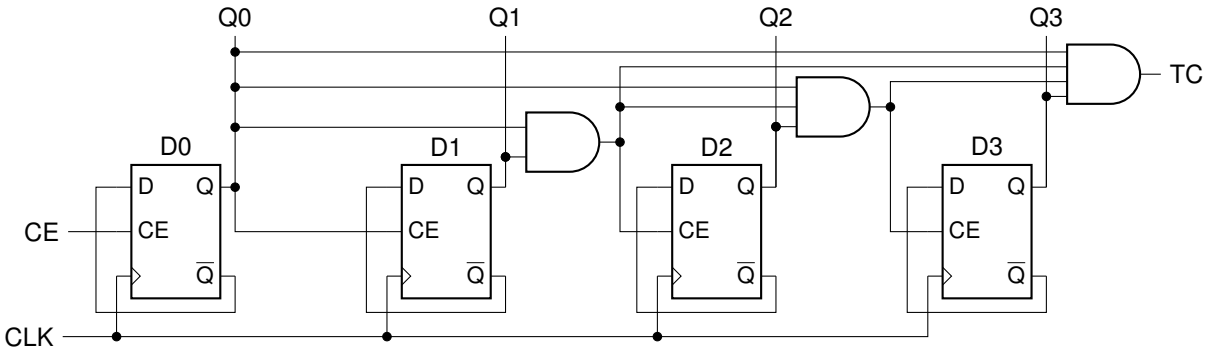


Figure 2.5: A 4-bit Synchronous Binary Counter

Note the additional complexity of this counter - we need to have a clock enable input, and we also need to be able to generate a terminal count output.

A general purpose synchronous counter in VHDL.

Where we want to display a number we generally don't want a binary output but instead a decimal output. We could convert binary to decimal in hardware using VHDL however it is easier, when using counters, to construct a BCD counter.

However, VHDL allows you to generalize your digital design using generic parameters. This allows flexibility. Instead of having a separate design for a binary counter and another for a BCD counter and another for - lets say a dice (which counts 1 to 6) we could produce a single design that can be reused for all of these.

In this exercise we will design a counter as shown in figure 2.6.

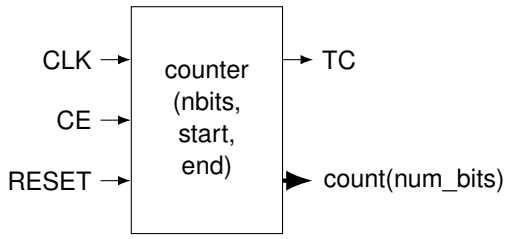


Figure 2.6: A general synchronous digital counter

For the reasons explained above we want to produce a synchronous counter. We will start by looking at a single stage. As explained above this stage must have a clock enable input and a terminal count output which true when the digit is at the end of its count.

The counter is required to count up in 1's between the given start and end numbers. It has an output count of num_bits bits which must be sufficient for the specified range. It also has an output TC (terminal count) which must be 1 when it is at the end of its range to enable it to be cascaded with other counters.

The counter has three inputs, the system clock, a clock enable - it will count on the rising edge of the clock only if clock enable is true and a reset which will reset it to its start value.

Exercise 6.

1. Complete the VHDL architecture in the provided template file `counter.vhd` for the counter shown in figure 2.6.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL; -- need this for std_logic types
use IEEE.NUMERIC_STD.ALL; -- need this for unsigned and signed types type

entity counter is
  generic (
    start: integer:=0; -- where count starts from
    finish: integer:=9; -- where count ends (inclusive)
    nbits: integer:=4 -- number of bits for counter
    );
  port (
    clk: in  std_logic;  -- system clock
    ce: in std_logic;    -- count or clock enable
    reset: in std_logic; -- asynchronous reset
    count: out unsigned(nbits-1 downto 0); -- output count
    tc: out std_logic:='0' -- set to '1' at terminal count for cascading
    );
end counter;
```

   In this template I have declared an internal counter **counter_i** to represent the register that will store the current value and two constants **ustart** and **ufinish** which are the start and finish values converted to unsigned types suitable for loading into the counter.



unsigned and signed types. The unsigned type is similar to std_logic_vector in that it represents an array of a specified number of bits however it also supports arithmetic and comparison operations such as addition and subtraction. The to_unsigned(<integer>,<nbits>) function will convert an integer into an unsigned type. There is also a signed type with similar operations. These are declared in the library IEEE.NUMERIC_STD.**ALL** which must be included at the start of the file. It is best practice to use signed or unsigned types when using VHDL for synthesis as it provides you with control over the number of bits used and the bit order.

2. Check your VHDL syntax by running a syntax check on it and if there are errors reported correct them.
3. Why is the clocked synchronous process sensitive to the clock and reset signals but not the clock enable signal?
4. How would you change the architecture to have a synchronous rather than asynchronous reset signal?
5. Is it possible to create an entity where the number of bits does not need to be declared as a generic parameter?
6. The provided testbench counter_testbench.vhd instantiates two cascaded counters and simulates them running through a complete cycle. View this VHDL file and make sure you understand it.

   What values do the counters count through? Identify where the clock period is set in the file.

2.3 Putting it all together - Hierarchical design and abstraction

In this section you will build a 4-BCD digit counter which will display its output on the LEDs. We will build this design up hierarchically starting using the VHDL entities you have previously designed and tested. This principle of hierarchical design is most important.

At the end of this practical you will learn how to:

1. represent a clocked entity in VHDL;
2. structural design in VHDL;
3. hierarchical design in VHDL;
4. the use of loops and **generate** to build arrays of components and connect them in VHDL;
5. implement and test your circuit on an FPGA.

Figure 2.7 shows the schematic for the counter you are to construct.

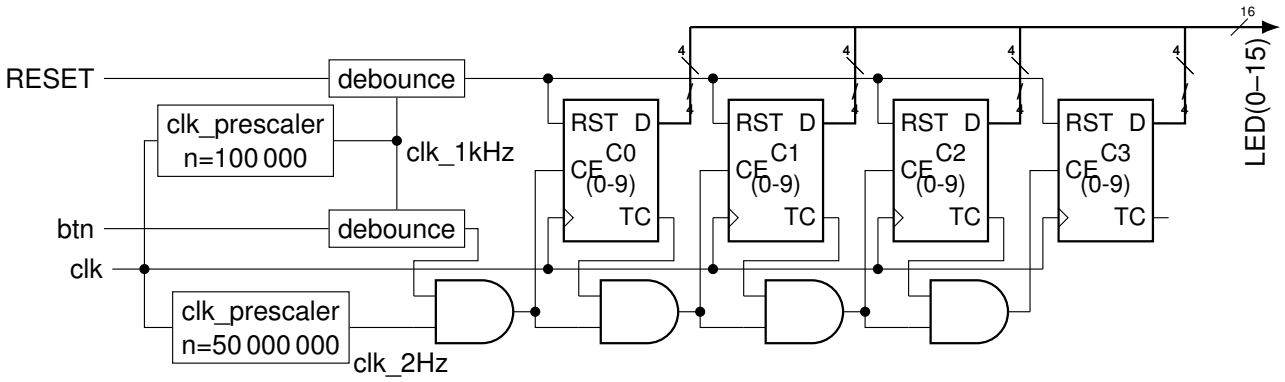


Figure 2.7: A fully synchronous 4-digit BCD counter

One of the useful features of using a language like VHDL is that you can easily represent repetition using loops and arrays of signals using vectors or arrays. In the BCD counter above we have four individual counter entities, each of which will have input signals and output signals.

To implement the 4-bit counter it is suggested that you will need to declare two internal vector signals of 4 elements: **tc** corresponding to the terminal count output of each counter and **ce** corresponding to the clock enable inputs of each counter.

We can represent the signals in VHDL using logic vectors, and we can instantiate the four counters (which I have numbered from as 0 which is the default first index in VHDL vectors) using a for loop and the generate statement.

Exercise 7.

1. Create a VHDL file `fourcounter.vhd` for your top level design. Declare the entity using the signal names shown in figure 2.7. Make sure you specify any libraries needed at the start.
2. Create a structural architecture to implement this entity. This should be entirely combinational - no processes needed. The basic steps you need to remember to do are
   - Provide component declarations for the clk_prescaler,debounce and counter entities.
   - Declare internal std_logic signals for internal 2Hz and 1kHz clocks, the debounced signals from the buttons and vectors to connect the ce and tc signals from the counters.
   - insert component instantiations for 2 prescalers and 2 debounce circuits
   - use a for generate loop to instantiate 4 counter entities and connect their ports to the appropriate signals
   - connect the counter tc and ce signals between counters using appropriate Boolean logic expressions.
3. Check your VHDL by running an integrity test on it and if there are errors reported correct them.
4. Make a copy of the constraints file `Basys3_Master.xdc` for this design renaming the copy `fourcounter.xdc`. Connect the inputs to buttons and the outputs to the 16 LEDs in the constraints file.

   You may need also to enable the clock by commenting out the lines

```
set_property PACKAGE_PIN W5 [get_ports clk]
create_clock -add -name sys_clk_pin -period 10.0 -waveform {0 5} [get_ports clk]
```

5. Edit the Makefile to add a dependency rule for `fourcounter.bit` which should include all the required VHDL files needed for this circuit.
6. Synthesize the circuit, program the development board and verify that it works as expected from your simulation.

By the end of this task, you should be confident of generating designs and test-benches, running simulations, configuring using a constraints file and programming an FPGA using the provided tool chain.

Chapter 3

Case Study: A Stopwatch

3.1 A Stopwatch

In this set of exercises you will implement a digital stopwatch in VHDL for synthesis on the Basys 3 development board with the following features.

- It will display lapsed time on the seven segment display with a 1/100 sec resolution, counting up to 99.99 seconds.
- A start/stop button starts and stops the timer. If a lap time is displayed this button will revert to displaying the current lapsed timer.
- A reset button will zero the timer and display.
- A lap button will freeze the displayed time but not stop the timer. Subsequent presses will display the new lap time when the button is pressed.
- If more than one button at a time is pressed this is invalid input and can be ignored.

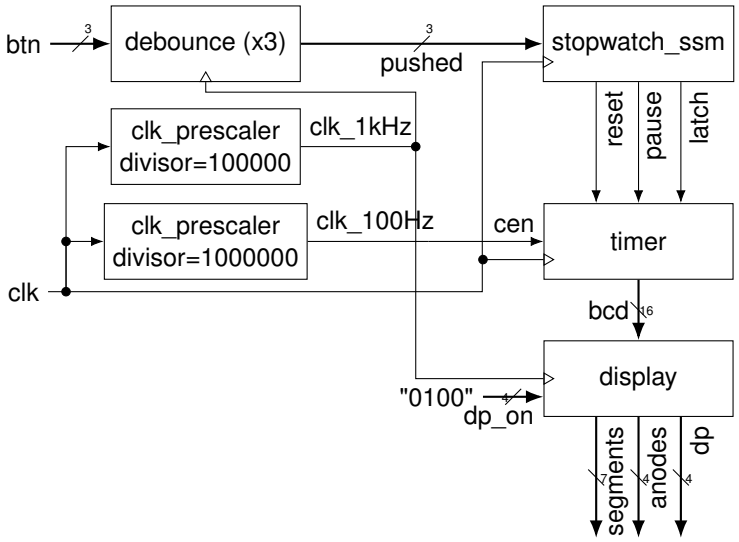


Figure 3.1: Block diagram of a digital stopwatch logic design

The structural design of the stopwatch you will implement is shown in figure 3.1.

The entities shown in this design are as follows

clk_prescaler Two instances are necessary to derive, from the 100 MHz master system clock, clock pulses at 100 Hz for the timer counter and a 1 kHz to strobe the display output and button input.

debounce Debounces the signals coming from the physical buttons to produce a clean digital signal. Three instances are necessary, one for each button.

btn_push Outputs a single pulse of one clock duration when a button is pressed [goes from 0 to 1]. Three instances of this are also necessary.

stopwatch_ssm Implements the main stop watch controller using a Moore model sequential state machine, taking the button push signals as inputs and outputing the control signals for the timer.

timer Uses control signals to manage its internal timer running at 100,Hz and outputs a 16-bit unsigned integer count for display. The three control signals are to control whether the timer is running or paused, whether the display is updated from the timer or not and to zero the timer.

display Takes a 4 BCD digit and decimal point input and provides the necessary multiplexing signals to drive the 7-segment display hardware.

You will develop these entities and test them in turn. You will need the counter, clk_prescaler and debounce entities you developed last term. In order to reduce duplication and errors in component definitions we provide a work package `IDD`. To use thee constants, components, functions created in the package file add the following to the start of your vhdl files

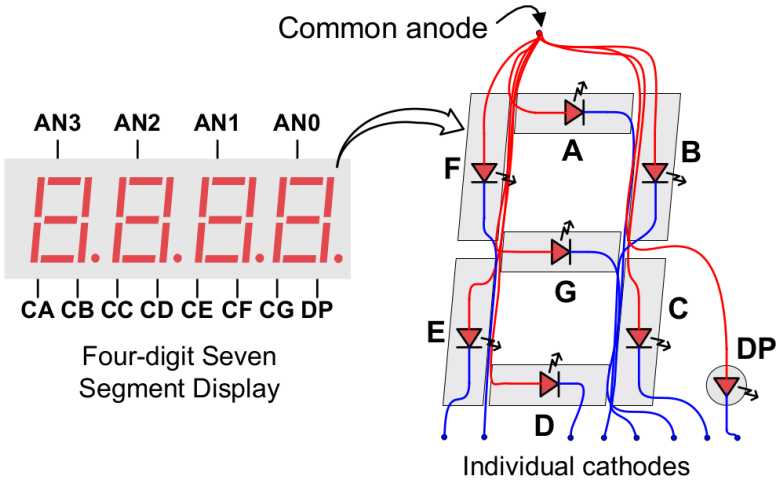
```
use work.IDD.all;
```

`work` refers to the current library - the files in the current working directory. `all` means import all definitions.

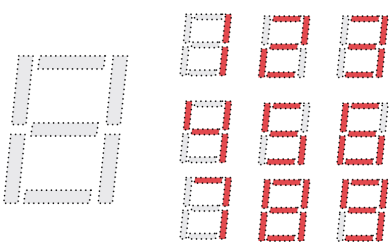
3.2 Seven-segment display

The Basys3 board contains one four-digit common anode seven-segment LED display. Each of the four digits is composed of seven segments arranged in a figure 8 pattern, with an LED embedded in each segment. Segment LEDs can be individually illuminated, so any one of 128 patterns can be displayed on a digit by illuminating certain LED segments and leaving the others dark, as shown in figure 3.2b. Of these 128 possible patterns, the ten corresponding to the decimal digits are the most useful.

The anodes of the seven LEDs forming each digit are tied together into one common anode circuit node, but the LED cathodes remain separate, as shown in figure 3.2a. The common anode signals are available as four digit-enable input signals to the 4-digit display. The cathodes of similar segments on all four displays are connected into seven circuit nodes labeled CA through CG (so, for example, the four D cathodes from the four digits are grouped together into a single circuit node called CD). These seven cathode signals are available as inputs to the 4-digit display. This signal connection scheme creates a multiplexed display, where the cathode signals are common to all digits, but they can only illuminate the segments of the digit whose corresponding anode signal is asserted.



(a) Common anode circuit



(b) Illumination patterns for decimal digits

Figure 3.2: The seven-segment display

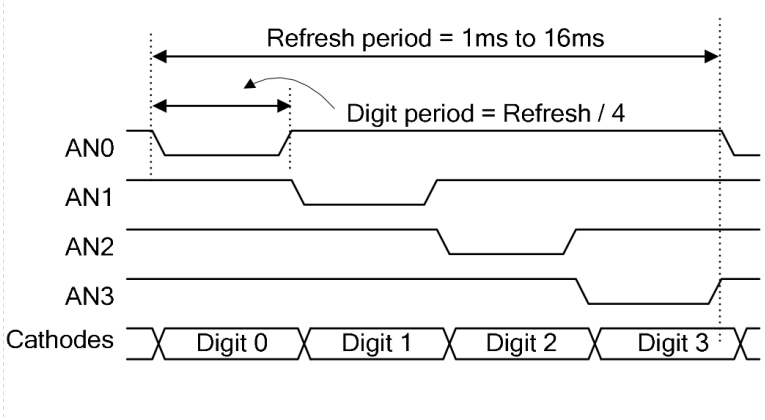


Figure 3.3: Four digit scanning display controller timing diagram

To illuminate a segment, the anode should be driven high while the cathode is driven low. However, since the Basys3 uses transistors to drive enough current into the common anode point, the anode enables are inverted. Therefore, both the AN0..3 and the CA..G/DP signals are driven low when active.

A scanning display controller circuit can be used to show a four-digit number on this display. This circuit drives the anode signals and corresponding cathode patterns of each digit in a repeating, continuous succession at an update rate that is faster than the human eye can detect. Each digit is illuminated just one-fourth of the time, but because the eye cannot perceive the darkening of a digit before it is illuminated again, the digit appears continuously illuminated. If the update, or refresh, rate is slowed to around 45 hertz, a flicker can be noticed in the display.

For each of the four digits to appear bright and continuously illuminated, all four digits should be driven once every 1 to 16 ms, for a refresh frequency of about 1 KHz to 60 Hz. For example, in a 62.5 Hz refresh scheme, the entire display would be refreshed once every 16 ms, and each digit would be illuminated for 1/4 of the refresh cycle, or 4 ms. The controller must drive low the cathodes with the correct pattern when the corresponding anode signal is driven high. To illustrate the process, if AN0 is asserted while CB and CC are asserted, then a “1” will be displayed in digit position 1. Then, if AN1 is asserted while CA, CB, and CC are asserted, a “7” will be displayed in digit position 2. If AN0, CB, and CC are driven for 4 ms, and then AN1, CA, CB, and CC are driven for 4 ms in an endless succession, the display will show “71” in the first two digits. An example timing diagram for a four-digit controller is shown in figure 3.3.

In this section you will design a circuit which, given a 4 digit BCD number, a suitable clock and a decimal point pattern will produce the necessary signals to display the digits with decimal points on the display.

Exercise 8. Design a seven segment display driver

1. Create a file `display.vhd` and declare the display entity as follows:

```
entity display is
  port (
    clk: in STD_LOGIC;
    clk_en: in  STD_LOGIC; -- refresh clock enable must be <=1kHz

    -- Input BCD and decimal point signals
    bcd: in STD_LOGIC_VECTOR(15 downto 0):=(others=>'0'); -- 4 BCD digits
    dp_on: in STD_LOGIC_VECTOR(3 downto 0):="0000" ; -- dp on enable

    -- Outputs to connect directly to external display hardware
    segments : out STD_LOGIC_VECTOR (6 downto 0); -- 7 segments cathode drive
    dp: out STD_LOGIC; -- decimal point
    anodes   : out STD_LOGIC_VECTOR (3 downto 0) -- anode drive (0 is on)
    );
end display;

architecture behavioral of display is
  signal digit : STD_LOGIC_VECTOR(3 downto 0);
  signal anodes_i : STD_LOGIC_VECTOR (3 downto 0) := "1110";
```

```
begin
  anodes <= anodes_i;

  digit_to_segment: process(digit)
  begin
    case digit is
      -- Cathodes                    GFEDCBA
      when "0000" => segments <= "1000000";
      -- complete for other digits.
      when others => segments <= "1111111"; -- all off if >9
    end case;
  end process;

  multiplex: process(clk)
  begin
    if rising_edge(clk) and clk_en='1' then
      -- go through the BCD digits in turn shifting the anode enable signal
    end if;
  end process;

 display_proc: process(anodes_i)
  begin
    -- set digit and dp to subset of input bits
    -- depending on current anodes_i
  end process;

end behavioral;
```

This entity takes a drive clock enable at 1 kHz produced by 'clk_prescaler', the 4 BCD digits as a vector (16 bits), and a 4 bit pattern indicating which decimal points are to be turned on. It will output the drive signals for the cathode segments (as a vector in order 'GFEDCBA'), the decimal point cathode and the anode enabel sigal as previously described.

2. The first step in designing the drive is to provide a look-up table which, given a decimal digit will provide the necessary 7 bit pattern to drive the segments. The process 'digit_to_segment' above illustrates one approach to implementing a look-up table in VHDL. It updates the segments when the digit signal is updated and using a case statement to decide what pattern to use. Complete the case statement in this process for the other digit values using the information provided in figure 3.2. Note this process describes a combinational (unclocked) circuit.

3. The second clocked process 'multiplex' is the multiplexing signal driven by the clock. On each enabled clock pulse it must shift along the anodes signal to select a new digit.

   The VHDL signal

   ```
   signal anodes_i : STD_LOGIC_VECTOR (3 downto 0) := "1110";
   ```

   specifies an anode pattern which would illuminate the least significant digit. You can right shift this cyclically on every clock edge using the expression

   ```
   anodes_i <= anodes_i(2 downto 0) & anodes_i(3);
   ```

   This is a common idiom used to perform shift register operations in VHDL.

4. A third combinational process, 'display_proc' must select the next set of digit bits from the BCD input signal, and the decimal point value setting the digit to be displayed, anode and decimal point out accordingly.

   Complete this process setting the digit, anodes and decimal signals appropriately. Remember the display drive signals are active low.

5. Save your VHDL file `display.vhd` and check that the syntax is correct.

6. A self-checking testbench ‘display_testbench‘ entity defined in the corresponding file has been provided for you to test your display implementation under simulation. You should run this and correct your VHDL if it reports any errors.

7. The structural top level VHDL file ‘display_demo.vhd‘ together with the configuration file ‘display_demo.xdc‘ may be used to program your Basys3 board to produce an output on its display. Use the synthesis tools to program your board using these files together with your ‘display‘ and ‘clk_prescaler‘ implementations. You may need to edit Taskfile.yml or Makefile to declare the necessary dependencies between the bit file and VHDL files (as taught in the previous laboratory). What number is displayed on the Basys3 board when it is programmed with this implementation? Exam the file ‘display_demo.vhd‘ to see where the number is set in the code and comapre with what is displayed.



The testbench for your display code does not easily enable you to see if the output is in fact correct. It is much easier to see on an actual 7-segment display if it is working correctly than from viewing the simulation results. If your display logic is not working correctly you should create and run the testbench and check the signal timing on simulation.

3.3 The Timer and the Stopwatch Control Logic

Up to now we have developed and test the input and output logic for our stopwatch. What is left is the main stopwatch functionality itself. You will design this using two entities. The first is a sequential state machine which takes the button push inputs from the user and provides three control lines to the main timer itself.

The timer has an internal 16 bit count register and a 16 bit register which holds the count value to be output for display. It takes the system clock to clock its process state and a slower clock running at 100Hz to provide the necessary timing information for the stop watch.

The three control signals to the timer are as follows.

reset Sets both the internal and displayed count to zero.

pause If true does not update the internal counter i.e. stops the timer. If false the internal timer register is updated once every 1/100 sec.

latch If true then the external count is updated to equal the internal timer. If false it is not.

3.3.1 Stopwatch control state machine

Exercise 9.

1. Design and draw a Moore model state machine diagram to implement the control logic which takes the debounced and decoded inputs for the stopwatch (as described in section 3.1) and generates the required control signals as described above.



This input control signals are being asserted for one clock cycle when the user pushed the appropriate button. The ‘pause‘ and ‘latch‘ control signals however have to be asserted for longer than one clock cycle. The ‘reset‘ output only needs to be asserted for one clock cycle to reset the timer. It is therefore not as simple as copying the input signals to the output. You should be able to implement this with 5 states.

2. Implement your state machine in VHDL in the file `stopwatch_ssm.vhd` with the following entity definition. You may wish to use the finite state machine template provided in the file `fsm_template.vhd`.

```
entity stopwatch_ssm is
  port (
    -- system clock
    clk: in std_logic;
    -- start/stop, lap, reset control inputs from debounced and decoded buttons
    ss, lap, rst: in std_logic;
    -- output reset, pause and latch control signal signals for timer
    timer_reset, timer_pause, timer_latch: out std_logic
  );
end stopwatch_ssm;
```

3. Open up the provided file `stopwatch_ssm_testbench.vhd`. This is a template for you to complete to test your stopwatch control logic state machine. Examine and make sure you understand this code.

   The VHDL function

   ```
   function to_string (a : std_logic_vector) return string is
   ```

   converts an input logic vector to a string. It is used to produce a display input and output signals in the testbench. This is an often useful function that is not part of the standard IEEE VHDL library. You may find it useful to keep in your VHDL arsenal.

   A VHDL procedure

   ```
   procedure check_response(
     signal stim_v: out std_logic_vector(2 downto 0);
     stim:  in std_logic_vector(2 downto 0);
     signal resp_v:  in std_logic_vector(2 downto 0);
     resp:  in std_logic_vector(2 downto 0)
   )
   ```

   is provided to set the stimulus signal to a specified value, waits one clock cycle and then check if the response signal is equal to a given expected response. It asserts an error and reports the values if not. This is something we often want to do repeatedly in a test bench and so it is appropriate to create a procedure.

   The test bench architecture has three processes. One loops round creating a clock signal signalled to finish. One asserts a clear signal for a short period at the start (this isn't needed in this case). The final process stim_proc is where you put in your testbench test cases.

4. Complete this testbench by adding in further tests. Make sure you check all possible transitions on your state diagram. Run the test bench simulation and correct your state machine logic as necessary.

3.3.2 Timer logic

Exercise 10.

1. Create your final VHDL architecture for the timer logic which should operate as per the description at this section. It has the following entity declaration.

   ```
   entity timer is
     port (
       cen: in std_logic; -- clock enable at 100 Hz tick frequency
       clk: in std_logic; -- master clock
       reset,pause,latch: in std_logic; -- reset count
       bcd: out std_logic_vector (15 downto 0) -- output BCD count to be displayed
     );
   end timer;
   ```

   You will need an internal counter signal in addition to the count output to implement storing lap times.

2. Construct a test bench to check that your timer is operating as expected. You should test for all possible inputs and that the output count increases as expected.

   I would recommend that for simulation purposes you generate the cen slow clock using clk_prescaler dividing down by only a small number (perhaps 2 or 3) to speed up the simulation.
   Rather than user asserts in your VHDL you could just look at the waveform output from the simulation - this may be simpler.

3.4 Putting it all together - the top level design and synthesis

By now you have developed and testing all the building blocks necessary to construct the top level circuit entity as shown in figure 3.1.

Exercise 11.

1. The top level stopwatch takes input from three push buttons and outputs the signals to drive the seven-segment display. Create the file `stopwatch.vhd` and write the entity definition necessary for this.
2. Create a constraints file `stopwatch.xdc` for this circuit making sure the signal names match those in your entity definition.
3. Now complete the implementation by writing a purely structural architecture implementation for this circuit as described in figure 3.1.

   Remember you will need two clk_prescaler entities to produce 1 kHz and 100 Hz clocks to strobe the i/o and driver the timer.
   You will also need three instances of the debounce and btn_push logic for the three switches.
   You can use the VHDL **for** ... **generate** loop to instantiate the debouncing and button push logic and connect them up rather than write out three separate similar blocks of VHDL.
4. Synthesize your design and use it to program the Basys 3 board. Test it out.
5. When you are satisfied it is operating correctly (it should if you tested all the intermediate circuits first) ask a lab demonstrator to assess it.

Appendix A

Some VHDL Style Guidelines

- When creating synthesizable code (RTL), you should write two types of code: behavioural RTL (leaf-level logic inference, sub-blocks) and structural code (blocks) – each exclusively in its own architecture.
- It is important to use a consistent, universal style for such things as entity declarations, component declarations, port mappings, functions, and procedures.
  - For signal assignments, port mappings and component instantiations, the general rule is one line per signal. You can exclude small components, but this rule will increase the readability and understandability of your code.
  - You should explicitly declare the port mappings. It makes it easier to understand the mapping and easier to alter your code later.
- Liberal comments are mandatory to maintain reusable code. Although VHDL is sometimes considered to be self-documenting code, it requires liberal comments to clarify intent, as any VHDL user can verify
  - *Commenting entities* can be an important tool for your design. Knowing exactly what behaviours are (and aren't) taken care of by an entity can help improve your design as well as its readability. This should be done using a comment header.
  - *Commenting processes and functions* is an important aspect of documenting your code, and should be done with a header describing the responsibility of that block of code.
  - *Commenting statements* within processes and functions is important so that the reader may understand not only that you have made a signal assignment, but also your reasoning behind it with respect to the entire code block. You need not comment every statement, but those that are key to the behaviour of the component or where the reasoning is not obvious from the code itself should generally be commented.
- Proper indentation ensures readability and reuse. Therefore, a consistent style is warranted. Many text editors are VHDL-aware, automatically indenting for 'blocks' of code, providing consistent indentation. Emacs has this capability. Proper indentation greatly simplifies reading the code. If it is easier to read, it is less likely that there will be coding mistakes by the designer.

  As for when to indent, the simple rule of thumb is that any chapter of code that is marked by an `end` (and, optionally, a `begin`) keyword should have its body indented.

  It is recommended that you use a good text editor that can automate the indentation and formatting of code and ensure consistency.
- Naming conventions maintain a consistent style, which facilitates design reuse. If all designers use the same conventions, designer A can easily understand and use designer B's VHDL code
  - *Processes* should have descriptive names to make clear their purposes and make it easier to identify problematic lines in simulation.
  - *Enitities, Architectures, Functions*
    - Names should describe, in some way, what the block of code does.
    - Names should be lowercase and use underscores to separate words.

[1] From http://webdocs.cs.ualberta.ca/~amaral/courses/329/labs/VHDL_Guideline.html

    - Entities should have unique names. Architecture need not have unique names as they are linked to entities, but should have a name resembling their entity (e.g. entity_arch or entity_rtl).
  - *Signal Naming Conventions*
    - Once again, use lowercase names with underscores.
    - Use <signal_name>_addr for addresses and <signal_name>_clk for clocks, for active-low signals, using <signal_name>_l is preferred, do not use _in and _out suffixes for port signal names. when these are connected to other signals with similar suffixes, your code can become very confusing.
    - You should use _i as a suffix when using a signal that is an internal representation of a port. (e.g. if 'output' is your port, output_i is the internal signal representing it).

- The most common problem (with signals) is that signals can be various data types. The problem in VHDL is "casting" from one data type to another. Unfortunately, no single function can automatically cast one signal type to another. Therefore, the use of a standard set of casting functions is important to maintain consistency between designers. Use the IEE.numeric_std library functions.

From	To	Function
std_logic_vector	unsigned	unsigned(std_logic_vector)
unsigned	integer	to_integer(unsigned)
integer	unsigned	to_unsigned(integer, no_of_bits)
unsigned	std_logic_vector	std_logic_vector(unsigned)

- The rules for signals are not complex

  Inverted Signals Whenever possible, use *active high* signals. This makes your code easier to understand and leads to better consistency.

  Entity Ports – Inputs can only be read, they cannot be assigned.
  – Outputs can only be assigned, they cannot be read.
  – Inout ports are useful when signals need to be read and assigned, but are used very rarely (e.g. data lines for RAM when reading and writing must occur).

  Internal Signals – A signal should only be assigned in one process.
  – For unclocked processes, never assign to a signal more than once.
  – For clocked processes, never assign a signal outside the reset or clock edge statements.

  Note: the above are guarded against by synthesis tools. If you have done any of the above, your design will not implement.

  Process Sensitivity List – For a combinatorial process, all signals that are read should be included in the sensitivity list.
  – For a clocked process, you need only the clock and reset signals in the sensitivity list.

- Variables are commonly not understood and are therefore not used. Variables are also commonly used and not understood. Variables can be very powerful when used correctly. This warrants an explanation of how to properly use variables.

  When combinatorial signals are to be used within a process, most often a designer will use variables. Variables, however, are treated quite differently than signals during synthesis and implementation.

  Variables are not synthesized necessarily as physical wires (whereas signals are). As such, their assignments are not purely combinatorial. Assignment to a variable depends on its order in a sequence of statements. The following is two uses of variables:

Correct Variable Assignment	Incorrect Variable Assignment

`variable := a and b;`
`c <= variable or e;`
`d <= variable or f;`
	`c <= variable or e;`
	`d <= variable or f;`
	`variable := a and b;`

  The assignment on the left writes to the variable before reading from it. As a result, the variable simply becomes a wire connecting the output of an and gate to the inputs of two or gates. The assignment on the right, however, reads the variable before writing to it. This tells the synthesis tool that the value being read is the previous value of the variable. As a result, the value is stored sequentially and this leads to a 'latch inferred' warning.

- In a synchronous design, only one clock and one edge of the clock should be used.

  Generating internal clocks or gating clock signals is generally problematic because of glitching and clock skew problems. Any clocked process should include an asynchronous reset signal to generate initial values for signals that are being set synchronously.

- Coding for Finite State Machines (FSM) includes analysing several tradeoffs.
  - In general, three processes are recommended for a state machine: next-state decoding, output decoding and registering state bits and outputs.
  - Always explicitly define behaviour for all states and outputs. If not, the results of synthesis may produce unexpected results.
  - It is generally suggested that one-hot encoding be used for state machines. For example, a 5 state machine should use 5 bits to encode the states. Each state should have one bit high and the others low.
- If-then-else and case statements can cause unwanted effects in a design. Specifically, nested if-the-else and case statements may cause extra levels of logic inference. This occurs because if-then-else statements generally infer priority-encoded logic. However, one level of an if-then-else will not necessarily create priority-encoded logic. For that matter, synthesis tools generally handle if-then-else or case statements very well and create parallel logic rather than priority-encoded logic.
  - Avoid, when possible, nesting conditional (if-then-else or case) statements.
  - Try to create exclusive conditional cases, rather than overlapping ones. This will limit any unintentional priority-encodings.
  - Always make explicit all cases (with else or when others condition to eliminate unwanted logic).